Python Programming Fundamentals

VS Code Debugger

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1.1 Beyond `print()` debugging

`print()` is useful, but limited:

- You have to guess where to put them
- Output clutters the terminal
- Hard to inspect complex objects

The debugger lets you:

- **Pause** execution at any line
- **Inspect** variable values interactively
- **Step through** code line by line
- **Evaluate** expressions on the fly

Outline

2.1 Click in the gutter

To set a breakpoint in VS Code:

1. Open your Python file
2. Click in the **left gutter** (grey area) next to a line number
3. A **red dot** appears — this is your breakpoint
4. Press **F5** (or Run → Start Debugging)

Execution will **pause** at the breakpoint, and VS Code enters debug mode.

Outline

3.1 What you see when paused

Panel	Shows
Variables	All variables in current scope + their values
Watch	Expressions you add to monitor
Call Stack	Which functions are currently active
Breakpoints	List of all set breakpoints

The **highlighted line** is the next line that will execute.

Outline

4. Step Controls

Button	Key	Action
▶ Continue	F5	Run until next breakpoint
⤵️ Step Over	F10	Run current line, stay in this function
⬇️ Step Into	F11	Jump **into** a function call
⬆️ Step Out	Shift+F11	Run to end of current function
⏹ Stop	Shift+F5	End debugging session

Step Over is the most-used — walk through your code one line at a time.

Outline

5.1 Monitor specific values

In the **Watch** panel, click + and type any Python expression:

- `shop.get_all_products()`
- `len(products)`
- `p.get_price() > 2.0`
- `product.get_name().upper()`

The expression is re-evaluated **at every pause**, so you can see how values change.

Outline

6.1 Interactive Python at a breakpoint

The **Debug Console** (bottom panel) lets you type Python while paused:

```
> shop.get_all_products()
[<product.Product object at 0x...>, ...]

> len(shop.get_all_products())
3

> shop.find_by_name("Apple")
Product [101]: Apple — €1.99
```

This is extremely useful for testing assumptions about object state.

Outline

7. Debugging a CRUD Bug

Example: `find_by_name()` always returns `None`.

1. Set a breakpoint inside `find_by_name()`
2. Run the debugger and call the method
3. **Step Over** each line and watch the Variables panel
4. Inspect `p.get_name()` and `name` in the Debug Console:

```
> p.get_name()
'Apple '          # ← trailing space!
> name
'Apple'
```

Fix: add `.strip()` to `get_name()` or when comparing.

Outline

8. launch.json — Debug Configuration

VS Code creates `.vscode/launch.json` automatically. The default Python config:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Python: Current File",
            "type": "python",
            "request": "launch",
            "program": "${file}",
            "console": "integratedTerminal"
        }
    ]
}
```

8. launch.json — Debug Configuration

This runs whichever file is currently open. You can add more configurations for specific entry points.

Thanks for Watching - Any questions?